
How Global Prayer Times Calculates Prayer Times

Solar geometry, calculation conventions, high-latitude rules,
and Gregorian–Hijri conversion

Public technical specification

Version 1.0 – generated from the behavior implemented in the global bot

github.com/escalopa/prayer-bot

Contents

1	Scope and source of truth	2
2	End-to-end calculation pipeline	2
3	Solar geometry	2
3.1	Prayer event definitions	3
4	Supported calculation methods	3
5	Asr and madhab selection	4
6	High-latitude rules	4
7	Corrections, precision, and caching	5
8	Gregorian and Hijri dates	5
8.1	Gregorian date to Julian day	5
8.2	Umm al-Qura lookup used by the bot	5
8.3	Arithmetic fallback outside the table	6
9	Why another timetable may differ	6
10	References	6

1 Scope and source of truth

This document explains the calculation performed by the *global* Telegram bot. It is a technical description, not a religious ruling. Local mosques and recognized religious authorities may publish schedules that include safety margins, observational decisions, or conventions different from the options available here.

The runtime source of truth is:

- `global/internal/prayertime/calculator.go` for prayer-time configuration;
- `github.com/hablullah/go-prayer` version 1.1.1 for solar-event and high-latitude logic;
- Go SAMPA version 1.0.0 for the NREL Solar Position Algorithm (SPA);
- `global/internal/hijri/calendar.go` and Go Hijri version 1.0.2 for calendar conversion.

The bot calculates prayer times locally. Google Maps is used only when a location is set, to resolve the IANA timezone, country, city label, and place identifier. Google does not supply the prayer times.

2 End-to-end calculation pipeline

For a user at latitude ϕ , longitude λ , and IANA timezone Z , the bot applies the following pipeline:

1. Accept Telegram coordinates and validate $-90 \leq \phi \leq 90$ and $-180 \leq \lambda \leq 180$.
2. Resolve Z and the ISO country code. Persist coordinates rounded to three decimal places:

$$\phi_s = \frac{\text{round}(1000\phi)}{1000}, \quad \lambda_s = \frac{\text{round}(1000\lambda)}{1000}.$$

3. Select a country-based default calculation method. The user may change it later.
4. Convert the requested instant into timezone Z and take that local civil date.
5. Calculate a full year of solar events for (ϕ_s, λ_s, Z) and cache it by year and settings.
6. Select Fajr, sunrise, Dhuhr, Asr, Maghrib, and Isha for the requested local date.
7. Apply the chosen high-latitude fallback when normal twilight is unavailable.
8. Apply per-prayer corrections and round each non-zero result to the nearest minute.
9. Convert the same local Gregorian date to a displayed Hijri date; this conversion never moves a prayer time.

Timezone conversion uses the IANA database, so daylight-saving changes and historical offset rules come from the resolved timezone rather than from a fixed UTC offset.

3 Solar geometry

The underlying solar engine implements the NREL Solar Position Algorithm. SPA determines topocentric solar position from the instant, observer location, and astronomical time parameters. The prayer library then searches for transit and target solar elevations.

For intuition, if δ is solar declination, H is the local hour angle, and h is solar elevation, the central spherical relationship is

$$\sin h = \sin \phi \sin \delta + \cos \phi \cos \delta \cos H. \tag{1}$$

For a target elevation h_0 , the corresponding magnitude of hour angle is

$$H_0 = \arccos\left(\frac{\sin h_0 - \sin \phi \sin \delta}{\cos \phi \cos \delta}\right). \quad (2)$$

The morning event uses the solution before solar transit and the evening event uses the solution after transit. If the argument of arccos lies outside $[-1, 1]$, that elevation is not reached on that date; this is the mathematical reason a high-latitude fallback may be necessary.

Equations 1 and 2 summarize the event condition. The runtime does not use a low-order declination approximation: it delegates topocentric position and event solving to the SPA-based library.

3.1 Prayer event definitions

Fajr

The pre-transit instant at which solar elevation is $-\theta_F$, where θ_F is the method’s Fajr angle.

Sunrise

The SPA sunrise event. The bot passes elevation as zero, so the library uses its standard sea-level treatment.

Dhuhr

Solar transit, when the Sun crosses the local meridian. The bot adds no built-in delay; a user correction can add one.

Asr

A post-transit target elevation derived from the selected shadow factor; see Section 5.

Maghrib

The SPA sunset event.

Isha

The post-transit instant at solar elevation $-\theta_I$, except for Umm al-Qura, which uses 90 minutes after Maghrib.

4 Supported calculation methods

Only Fajr and Isha differ among these nine method presets. Sunrise, Dhuhr, Asr, and Maghrib retain the definitions above.

Method	Fajr	Isha	Runtime convention
Muslim World League (MWL)	18°	17°	Default outside explicitly mapped countries.
Egyptian General Authority	19.5°	17.5°	Default for Egypt.
Umm al-Qura	18.5°	90 min	Isha is exactly Maghrib plus 90 minutes.
University of Islamic Sciences, Karachi	18°	18°	Default for Pakistan, India, Bangladesh, and Afghanistan.
ISNA	15°	15°	Default for the United States and Canada.
Diyanet	18°	17°	Default for Turkey.
Kemenag	20°	18°	Default for Indonesia.

Method	Fajr	Isha	Runtime convention
MUIS	20°	18°	Default for Singapore.
JAKIM	20°	18°	Default for Malaysia.

For an angle-based method, the event conditions are

$$h(t_F) = -\theta_F, \quad t_F < t_{\text{transit}}, \quad h(t_I) = -\theta_I, \quad t_I > t_{\text{transit}}.$$

For Umm al-Qura, the final Isha rule is

$$t_I = t_M + 90 \text{ minutes}, \quad (3)$$

where t_M is Maghrib. This fixed-duration rule is applied after the general high-latitude adapter, so a selected night-fraction rule can adjust Umm al-Qura Fajr but does not replace its fixed Isha duration.

5 Asr and madhab selection

Let $k = 1$ for Shafi'i, Maliki, and Hanbali timing, and $k = 2$ for Hanafi timing. With topocentric solar declination δ for the day, the target Asr elevation is

$$h_A = \text{arccot}(k + \tan |\delta - \phi|), \quad \text{arccot}(x) = \arctan\left(\frac{1}{x}\right). \quad (4)$$

The solar-event solver finds the post-transit instant t_A at which $h(t_A) = h_A$. The $k = 2$ target is lower in the afternoon sky and therefore normally produces a later Asr time than $k = 1$.

6 High-latitude rules

The normal schedule is marked abnormal if sunrise or sunset is missing, or if astronomical dawn/dusk at -18° is missing. A configured adapter is then run. The three adapters exposed by this bot change a date only when Fajr or Isha is missing and both sunrise and Maghrib exist.

Define the daytime and nighttime durations

$$D = t_M - t_R, \quad N = 24 \text{ hours} - D, \quad (5)$$

where t_R is sunrise and t_M is Maghrib. Each rule chooses fractions p_F and p_I and sets

$$t_F = t_R - p_F N, \quad t_I = t_M + p_I N. \quad (6)$$

Rule	p_F	p_I
Angle based	$\theta_F/60$	$\theta_I/60$
Middle of the night	1/2	1/2
One seventh of the night	1/7	1/7

For example, with an eight-hour night and an 18° angle, the angle fraction is $18/60 = 0.3$, so the fallback places Fajr 2 hours 24 minutes before sunrise and Isha 2 hours 24 minutes after Maghrib.

These three rules require an actual sunrise and sunset. During polar day or polar night, when one of those events is absent, they cannot synthesize a complete schedule. The bot should not imply certainty in that case; users should follow an appropriate local authority.

7 Corrections, precision, and caching

Each prayer has a user correction Δ_p in whole minutes, constrained by the UI/API to $-30 \leq \Delta_p \leq 30$. The final value is

$$t_{p,\text{shown}} = \text{round}_{\text{minute}}(t_{p,\text{base}} + \Delta_p). \quad (7)$$

Rounding is to the nearest minute, not always downward. A correction affects only the selected prayer. The Hijri correction is separate and never changes these instants.

Schedules are calculated one local year at a time and cached by rounded coordinates, timezone, method, madhab, high-latitude rule, all six prayer corrections, and year. This cache changes performance only; it does not change the equations.

8 Gregorian and Hijri dates

The bot receives a civil instant, converts it to the user's local timezone, and uses that local Gregorian year-month-day. Before calendar conversion only, it applies a user moon-sighting correction $a \in \{-2, -1, 0, 1, 2\}$:

$$G' = G + a \text{ civil days}. \quad (8)$$

The prayer schedule remains attached to G .

8.1 Gregorian date to Julian day

For a Gregorian date with year Y , month M , day d , and fraction of day f , January and February are treated as months 13 and 14 of the preceding year. Let

$$(Y', M') = \begin{cases} (Y - 1, M + 12), & M \leq 2, \\ (Y, M), & M > 2, \end{cases}$$

and, for Gregorian dates,

$$A = \left\lfloor \frac{Y'}{100} \right\rfloor, \quad B = 2 + \left\lfloor \frac{A}{4} \right\rfloor - A.$$

The library's Julian-day expression is

$$JD = 1720994.5 + \lfloor 365.25Y' \rfloor + \lfloor 30.6001(M' + 1) \rfloor + B + d + f. \quad (9)$$

8.2 Umm al-Qura lookup used by the bot

The preferred conversion is a published table of Umm al-Qura lunation starts. The bundled table covers Gregorian dates from 14 March 1937 through 16 November 2077. The converter evaluates the adjusted date at UTC noon, calculates JD , and defines

$$CJDN = \lfloor JD \rfloor, \quad MCJDN = CJDN - 2400000. \quad (10)$$

Let L_i be the table of month-start MCJDN values and choose the first i for which $L_i > MCJDN$. Then

$$ILN = i + 16260, \quad (11)$$

$$y = \left\lfloor \frac{ILN - 1}{12} \right\rfloor + 1, \quad (12)$$

$$m = ILN - 12 \left\lfloor \frac{ILN - 1}{12} \right\rfloor, \quad (13)$$

$$d = MCJDN - L_{i-1} + 1. \quad (14)$$

The displayed Hijri date is (y, m, d) localized into the user's selected language.

For the inverse Umm al-Qura mapping, which the library supports but the bot UI does not currently call,

$$ILN = m + 12(y - 1), \quad (15)$$

$$i = ILN - 16260, \quad (16)$$

$$MCJDN = d - 1 + L_{i-1}, \quad (17)$$

$$JD = MCJDN + 2400000 - 0.5, \quad (18)$$

after which the standard inverse Julian-day algorithm produces the Gregorian date.

8.3 Arithmetic fallback outside the table

If G' falls outside the Umm al-Qura table, the bot uses the library's default tabular Islamic calendar. It has a 30-year cycle of 10,631 days, with leap years

$$2, 5, 7, 10, 13, 16, 18, 21, 24, 26, 29.$$

Months alternate between 30 and 29 days, starting with 30; month 12 has 30 days in a leap year. A convenient forward expression for Hijri (y, m, d) is

$$JD_H = 1948438.5 + 354(y - 1) + \left\lfloor \frac{3 + 11y}{30} \right\rfloor + 29(m - 1) + \left\lfloor \frac{m}{2} \right\rfloor + d. \quad (19)$$

The inverse counts complete 30-year cycles, then complete years and alternating months from

$$D = \lfloor JD - 1948438.5 \rfloor. \quad (20)$$

This arithmetic calendar is deterministic, but it is not a claim about observed crescent visibility.

9 What can cause differences from another timetable?

Two correct implementations can differ because of:

- a different twilight method or Asr shadow factor;
- high-latitude policy;
- elevation, refraction, or solar-position implementation;
- timezone or daylight-saving assumptions;
- institutional safety margins and per-prayer corrections;
- local crescent observation versus calculated Umm al-Qura dates;
- final rounding rules.

The bot stores coordinates only to three decimal places and uses zero elevation. At high latitudes, small changes in target angle can produce large clock-time differences because the Sun crosses the target elevation slowly.

10 Reproducibility and references

The following primary implementation and algorithm references correspond to the deployed dependency versions:

1. [Global bot prayer calculator source](#)
2. [Global bot Hijri wrapper source](#)
3. [Go Prayer v1.1.1 source](#)
4. [Go SAMPA v1.0.0 source](#)
5. [Go Hijri v1.0.2 source](#)
6. I. Reda and A. Andreas.
Solar Position Algorithm for Solar Radiation Applications.
NREL/TP-560-34302, revised 2008. [NREL report](#)

This document is maintained with the open-source bot. Corrections are welcome through GitHub issues or pull requests.